Kubernetes Ingress Controller with AWS ALB

Expanding on my previous post, today I'll walk you through setting up an **Ingress Controller** using Kubernetes services. Let's dive into this hands-on guide to expose Kubernetes services to the outside world.

Why Use AWS ALB for Ingress?

AWS recommends using the **Application Load Balancer (ALB)** over the Classic Load Balancer due to its advanced capabilities. ALB supports:

- **Host-based and path-based routing**
- **TLS (Transport Layer Security) termination**
- **WebSockets and HTTP/2**
- **AWS WAF (Web Application Firewall) integration**
- **Integrated access logs and health checks**

How the AWS ALB Ingress Controller Works

The **AWS ALB Ingress Controller** automatically provisions an ALB along with necessary AWS resources whenever an **Ingress resource** is declared in a Kubernetes cluster. This ALB then efficiently routes HTTP(S) traffic to various services within the cluster.

With its high availability and scalability, ALB is a great choice for managing ingress traffic. Using a single ALB, you can route traffic to multiple projects, optimizing costs and resource utilization.

Follow below steps to setup a simple ingress controller for nginx frontend application:

Step 1: Launch an EC2 Instance:

To deploy our infrastructure, we first need an EC2 instance. You can launch an instance via:

- **Terraform** (recommended for automation)
- **AWS Console** (for a manual setup)
- **Step 2: Configure eksctl and kubectl:**

Before creating an **EKS cluster**, configure `eksctl` and `kubectl` to interact with AWS.

```
ARCH=amd64
PLATFORM=$(uname -s)_$ARCH
```

```
#kubectl installation
curl -O https://s3.us-west-2.amazonaws.com/amazon-eks/1.32.0/2024-12-20/bin/linux/amd64/kubectl
chmod +x ./kubectl
mv kubectl /usr/local/bin/kubectl

#eksctl
curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$PLATFORM.tar.gz"
tar -xzf eksctl_$PLATFORM.tar.gz -C /tmp && rm eksctl_$PLATFORM.tar.gz
mv /tmp/eksctl /usr/local/bin
```

Step 3: Create an EKS Cluster

Create an EKS cluster using the following sample `eks.yaml` file:

```
3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/Docker-Install-tf.git
[ ec2-user@ip-172-31-31-95 ~/Docker-Install-tf ]$ eksctl create cluster --config-file=eks.yaml
2025-06-24 02:18:50 [ℹ]  eksctl version 0.210.0
2025-06-24 02:18:50 [ℹ]  using region us-east-1
2025-06-24 02:18:50 [ℹ]  skipping us-east-1d from selection because it doesn't support the following instance type(s): c3.large
2025-06-24 02:18:50 [ℹ]  skipping us-east-1f from selection because it doesn't support the following instance type(s): c3.large
2025-06-24 02:18:50 [ℹ]  setting availability zones to [us-east-1b us-east-1c]
2025-06-24 02:18:50 [ℹ]  subnets for us-east-1b - public:192.168.0.0/19 private:192.168.64.0/19
2025-06-24 02:18:50 [ℹ]  subnets for us-east-1c - public:192.168.32.0/19 private:192.168.96.0/19
2025-06-24 02:18:50 [ℹ]  nodegroup "expense" will use "" [AmazonLinux2023/1.32]
2025-06-24 02:18:50 [ℹ]  using Kubernetes version 1.32
2025-06-24 02:18:50 [ℹ]  creating EKS cluster "expense" in "us-east-1" region with managed nodes
2025-06-24 02:18:50 [ℹ]  1 nodegroup (expense) was included (based on the include/exclude rules)
2025-06-24 02:18:50 [ℹ]  will create a CloudFormation stack for cluster itself and 1 managed nodegroup stack(s)
2025-06-24 02:18:50 [ℹ]  if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=us-east-1 --cluster=expense'
2025-06-24 02:18:50 [ℹ]  Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "expense" in "us-east-1"
2025-06-24 02:18:50 [ℹ]  CloudWatch logging will not be enabled for cluster "expense" in "us-east-1"
2025-06-24 02:18:50 [ℹ]  you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=us-east-
1 --cluster=expense'
2025-06-24 02:18:50 [ℹ]  default addons metrics-server, vpc-cni, kube-proxy, coredns were not specified, will install them as EKS addons
2025-06-24 02:18:50 [ℹ]
2 sequential tasks: { create cluster control plane "expense",
    2 sequential sub-tasks: {
        2 sequential sub-tasks: {
            1 task: { create addons },
            wait for control plane to become ready,
        },
        create managed nodegroup "expense",
```

```yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: expense
  region: us-east-1

managedNodeGroups:
  - name: expense
    instanceTypes: ["m5.large", "c3.large","c4.large","c5.large"]
    desiredCapacity: 3
    spot: true
```

Step 4: Set Up OIDC for AWS and EKS To allow AWS services to interact with the EKS cluster, establish an OIDC (OpenID Connect) provider:

```
REGION_CODE=us-east-1
CLUSTER_NAME=expense
ACC_ID=3617695727

### Permissions

* OIDC provider
```
eksctl utils associate-iam-oidc-provider \
    --region $REGION_CODE \
    --cluster $CLUSTER_NAME \
    --approve
```

## Step 5: Configure IAM Policy and Attach It to the Cluster
```

Create an IAM policy for the **AWS Load Balancer Controller**:

```
* IAM Policy
```
curl -o iam-policy.json https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.12.0/docs/install/iam_policy.json
```
* Create IAM Policy
```
aws iam create-policy \
    --policy-name AWSLoadBalancerControllerIAMPolicy \
    --policy-document file://iam-policy.json
```
* Provide access to EKS through IAM Policy
```
eksctl create iamserviceaccount \
--cluster=$CLUSTER_NAME \
--namespace=kube-system \
--name=aws-load-balancer-controller \
--attach-policy-arn=arn:aws:iam::$ACC_ID:policy/AWSLoadBalancerControllerIAMPolicy \
--override-existing-serviceaccounts \
--region $REGION_CODE \
--approve
```
```

Step 6: Install AWS Load Balancer Controller Using Helm

```
[ ec2-user@ip-172-31-31-95 ~/Docker-Install-tf ]$ helm repo add eks https://aws.github.io/eks-charts
"eks" has been added to your repositories

3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/Docker-Install-tf.git
[ ec2-user@ip-172-31-31-95 ~/Docker-Install-tf ]$ helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=
$CLUSTER_NAME --set serviceAccount.create=false --set serviceAccount.name=aws-load-balancer-controller
NAME: aws-load-balancer-controller
LAST DEPLOYED: Tue Jun 24 02:51:24 2025
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
```

Now we can see the drivers are available under kube-system namespace. These drivers will create, manage and delete load balancers

```
3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/Docker-Install-tf.git
[ ec2-user@ip-172-31-31-95 ~/Docker-Install-tf ]$ kubectl get pods -n kube-system
NAME                                           READY   STATUS    RESTARTS   AGE
aws-load-balancer-controller-77cf9df776-5796w  1/1     Running   0          102s
aws-load-balancer-controller-77cf9df776-9g5t4  1/1     Running   0          102s
aws-node-9cdcb                                 2/2     Running   0          22m
aws-node-c4r8h                                 2/2     Running   0          22m
aws-node-cwjvz                                 2/2     Running   0          22m
coredns-6b9575c64c-9kgxb                       1/1     Running   0          26m
coredns-6b9575c64c-cpn22                       1/1     Running   0          26m
kube-proxy-hlptf                               1/1     Running   0          22m
kube-proxy-szd92                               1/1     Running   0          22m
kube-proxy-wgfqz                               1/1     Running   0          22m
metrics-server-579b5fcb66-5hm2t                1/1     Running   0          26m
metrics-server-579b5fcb66-7s2ld                1/1     Running   0          26m
```

Step 7: Create an Nginx Application and Build a Docker Image

Write a **Dockerfile** for an Nginx-based frontend, build and push the image to **Amazon Elastic Container Registry (ECR)** or **Docker Hub**:

```
k8s-ingress > app1 > Dockerfile
FROM nginx
RUN rm -rf /usr/share/nginx/html/index.html
RUN echo "<h1>I am from app-1</h1>" > /usr/share/nginx/html/index.html
```

Step 8: Deploy the Nginx App on EKS

Create a **Kubernetes manifest** (`manifest.yaml`) to deploy the application as pods (`app1`):

Also we will create service and attach it the pod using labels and selectors(app1).

```yaml
kind: Deployment
apiVersion: apps/v1
metadata:
  name: app1
  labels:
    app: app1
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app1
  template:
    metadata:
      labels:
        app: app1
    spec:
      containers:
      - name: app1
        image: lakshman143/app1:1.0.0
        ports:
        - containerPort: 80
---
kind: Service
apiVersion: v1
metadata:
  name: app1
spec:
  selector:  #labels are used as selectors
    app: app1
  ports:
  - protocol: TCP
    port: 80    #service port
    targetPort: 80
```

Step 9: Set Up AWS ALB Ingress Controller

Define an **Ingress resource** to route external traffic using AWS ALB:

- **IP-Based Targeting**: The ALB is set as **internet-facing**, and we specify `target-type: ip`, allowing pod IPs to be dynamically attached to the load balancer.
- **Internet-Facing Configuration**: Since the ALB needs to be accessible externally, we configure it to listen on **HTTPS (port 443)** and associate it with an **SSL certificate ARN** for secure connections.
- **Group Name**: We assign the ALB to a specific **Ingress group** (`awsalbingress`), enabling multiple services to share the same load balancer.
- **Host-Based Routing**: Requests made to **app1.devopsaws82s.online** are routed to the **app1 service** on **port 80**. The service then forwards traffic to the appropriate pod based on labels and selectors.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app1
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS": 443}]'
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-1:503561459301:certificate/3b1fc553-c58f-4145-8395-6d7386479c19
    alb.ingress.kubernetes.io/group.name: joindevops
spec:
  ingressClassName: alb
  rules:
  - host: "app1.lakshman.site"
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: app1
            port:
              number: 80
```

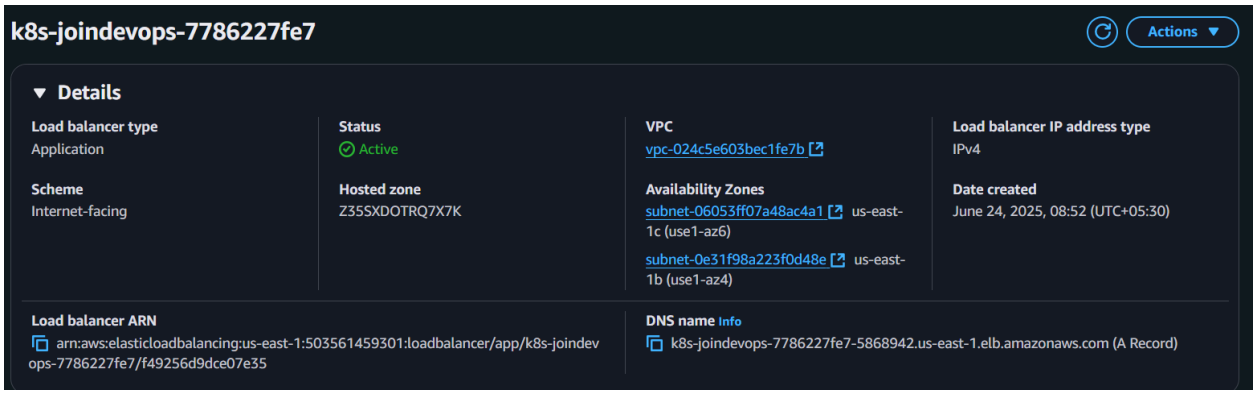
After executing steps 8 and 9, pod named app1 and svc named app1 got created.

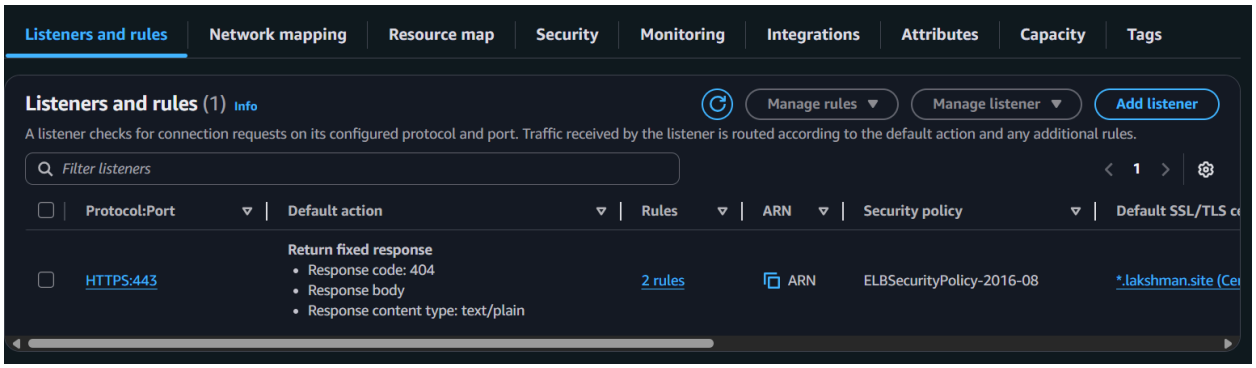
```
3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/k8-ingerss.git
[ ec2-user@ip-172-31-31-95 ~/k8-ingerss/app1 ]$ kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
app1-ff755d475-gmkmv    1/1     Running   0          8s
app1-ff755d475-lmfwx    1/1     Running   0          8s

3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/k8-ingerss.git
[ ec2-user@ip-172-31-31-95 ~/k8-ingerss/app1 ]$ kubectl get svc
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
app1         ClusterIP   10.100.248.106   <none>        80/TCP    24s
kubernetes   ClusterIP   10.100.0.1       <none>        443/TCP   35m
```

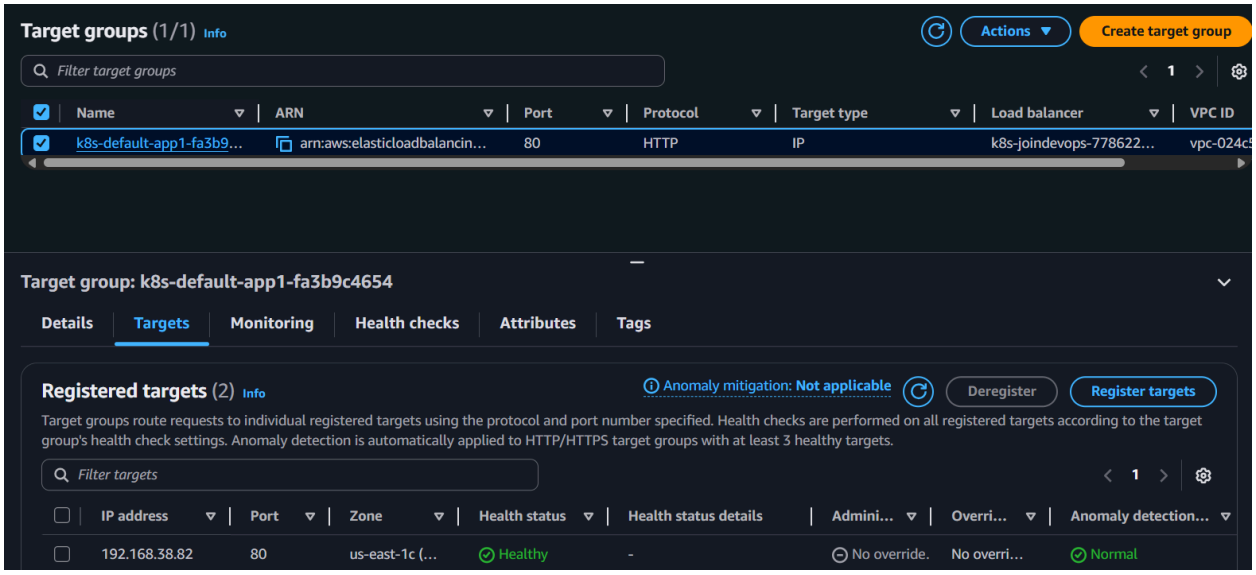
Also a ingress resource is created along with ALB.

```
3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/k8-ingerss.git
[ ec2-user@ip-172-31-31-95 ~/k8-ingerss/app1 ]$ kubectl get ingress
NAME   CLASS   HOSTS               ADDRESS                                                   PORTS   AGE
app1   alb     app1.lakshman.site  k8s-joindevops-7786227fe7-5868942.us-east-1.elb.amazonaws.com   80      7s
```

Now the ingress resource will create the ALB, Listener, rules and target group as shown below. Ingress is attached to service so it can fetch the pods and add them to target group.







Step 10: Deploy `app2` and Attach It to the Same ALB

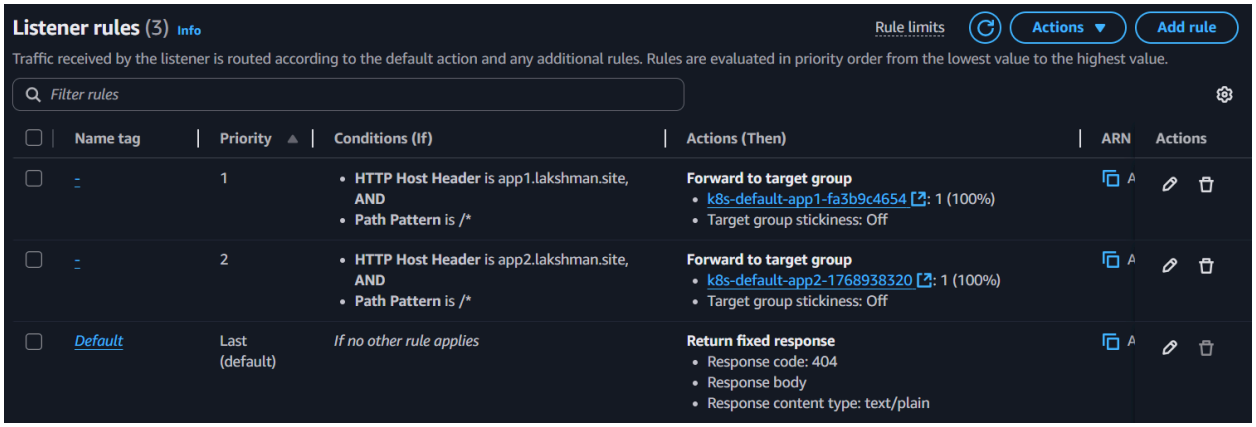
Repeat **steps 7, 8, and 9** for `app2`. Since we have specified the same `group.name`, `app2` will be attached to the existing ALB.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app2
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS": 443}]'
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-1:503561459301:certificate/8a1d5676-c3db-4bb9-ba6a-98580c36fd0b
    alb.ingress.kubernetes.io/group.name: joindevops
spec:
  ingressClassName: alb
  rules:
  - host: "app2.lakshman.site"
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: app2
            port:
              number: 80
```

Now the app2 pod and app2 service have been created.

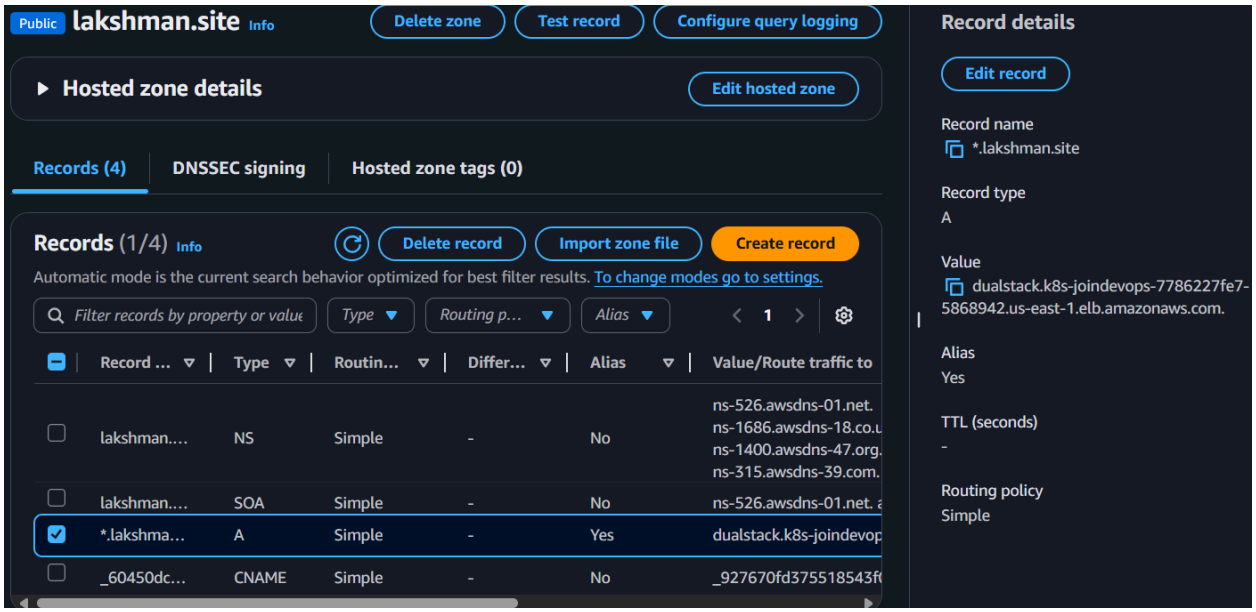
```
3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/k8-ingerss.git
[ ec2-user@ip-172-31-31-95 ~/k8-ingerss/app2 ]$ kubectl apply -f manifest.yaml
deployment.apps/app2 created
service/app2 created
ingress.networking.k8s.io/app2 created

3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/k8-ingerss.git
[ ec2-user@ip-172-31-31-95 ~/k8-ingerss/app2 ]$ kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
app1-ff755d475-s4vzk    1/1     Running   0          10m
app1-ff755d475-wpvt5    1/1     Running   0          10m
app2-56f664fc5f-mhwhp   1/1     Running   0          9s
app2-56f664fc5f-pfl7n   1/1     Running   0          9s

3.91.48.158 | 172.31.31.95 | t3.micro | https://github.com/Lakshmanp145/k8-ingerss.git
[ ec2-user@ip-172-31-31-95 ~/k8-ingerss/app2 ]$ kubectl get svc
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
app1         ClusterIP   10.100.138.243   <none>        80/TCP    11m
app2         ClusterIP   10.100.73.56     <none>        80/TCP    58s
kubernetes   ClusterIP   10.100.0.1       <none>        443/TCP   69m
```

Now, when we check the **Ingress resource**, we should see an **additional rule** added for `app2`.



Step 11: Configure Route 53 for Domain Mapping

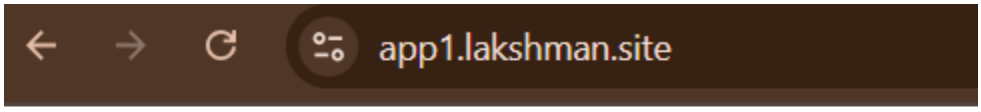
Update your **Route 53** records to map the **ALB DNS name** to your domains:



Step 12: Verify Routing

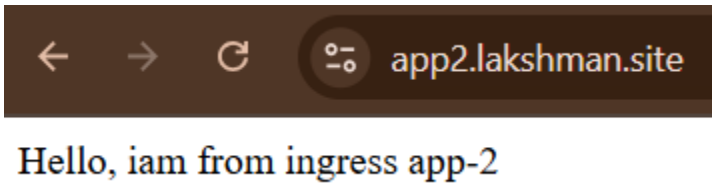
Once everything is set up, test the application:

When we hit https://app1.lakshman.site/ it will redirect as below



Hello iam from ingress app1

When we hit https://app2.lakshman.site/ it will redirect as below



Conclusion

By following these steps, we have successfully:

✅ Deployed an EKS cluster and installed AWS ALB Ingress Controller

✅ Created and exposed **Nginx-based frontend applications** (`app1` and `app2`)

✅ Configured **host-based routing** with a **single ALB**

✅ Mapped custom domains using **Route 53**

This setup is **cost-efficient**, **scalable**, and **production-ready** for managing multiple applications with a shared **Application Load Balancer**.

Let me know in the comments if you found this helpful!

Please fork my GitHub repository k8s-ingress and implement the necessary changes.